

Project Report

Project Name: Purrward www.purrward.app

Reference ID: IBF2O336

1. Executive Summary

In 2-3 sentences, summarize what your project does and the impact or value it delivers. Write this last, once everything else is done.

Purrward is a mobile-first cat wellness app that helps users track daily cat care tasks, verify care activity, and earn Purrpoints for consistent routines. The project supports both personal pet cats and community cats by making care easier to remember, easier to prove, and more rewarding through a simple care-to-earn system.

2. Project Overview

2a. Why you're building what you're building

Describe the problem you're solving and the motivation behind your project. What gap or need does it address?

Purrward is built to help people take better care of cats, whether they are personal pets or community cats. Cat care includes small daily routines such as feeding, giving water, cleaning litter, giving medicine, and checking the cat's condition. These tasks are important, but they can be easy to forget when people are busy.

The project addresses this by turning cat care into a simple reminder and reward system. Users can complete care tasks, track their progress, and earn rewards such as coupons. This can help users save money while also encouraging them to care for cats more consistently.

The motivation behind Purrward is to make cat care feel easier, more organized, and more rewarding. Instead of only reminding users what to do, the app gives them a reason to keep building better care habits.

2b. How it relates to the theme

Explain how your project connects to the hackathon theme.

Purrward relates to the hackathon theme because it focuses directly on improving the lives of cats. The app is designed around feline wellness, daily care, and responsible pet ownership.

The project also supports both pet cats and community cats. This makes the idea more useful beyond only individual pet owners. By rewarding care actions, Purrward encourages people to feed, monitor, and support cats more regularly.

2c. Target Audience

Who is this project for? Describe the user(s) or use case it's designed to serve.

Purrward is designed for cat owners, cat lovers, and people who regularly care for community cats. The main users are people who want help remembering and tracking daily cat care routines.

This includes students, busy pet owners, families, shelter volunteers, and people who feed or monitor stray/community cats. The app is also useful for users who want to save money through reward coupons while still building better care habits.

The app is made to be simple and mobile-friendly, so users do not need technical knowledge to use it.

3. Key Features

List the major core features you built:

☒ Cat care task tracker and reminder

Users can track daily care activities such as feeding, water, litter, medicine, and general wellness tasks.

☒ Care-to-earn reward system

Users earn Purrpoints after completing care tasks, which can later be connected to coupons, rewards, or other benefits.

☒ Cat profile and care experience

Users can manage care for pet cats or community cats through a friendly cat-focused interface with avatar-based feedback.

4. Technology Stack

List the languages, frameworks, libraries, and platforms used. Layer is something like (frontend, backend, database, etc)

Layer	Technology
Frontend (Client)	• Framework: SvelteKit 5 (using Svelte Runes) • Styling: TailwindCSS v4 • Icons: Lucide Svelte (@lucide/svelte) • Onboarding: driver.js (interactive product tours) • Internationalization: Paraglide JS (@inlang/paraglide-js) • Content / Markdown: MDsvex (mdsvex), Shiki (syntax highlighting), Gray-matter, Rehype plugins • Utilities: Fuse.js (fuzzy search)
Backend / API	• Framework/Hosting: SvelteKit on Cloudflare Pages / Workers • Validation: Zod (zod) • Language: TypeScript
Database / Storage	• Database: Turso (LibSQL) • ORM: Drizzle ORM • Migration Tool: Drizzle Kit
External APIs / Integration	• AI Platform: Gemini API (for photo verification and AI vet triage) • Authentication: Google OAuth 2.0

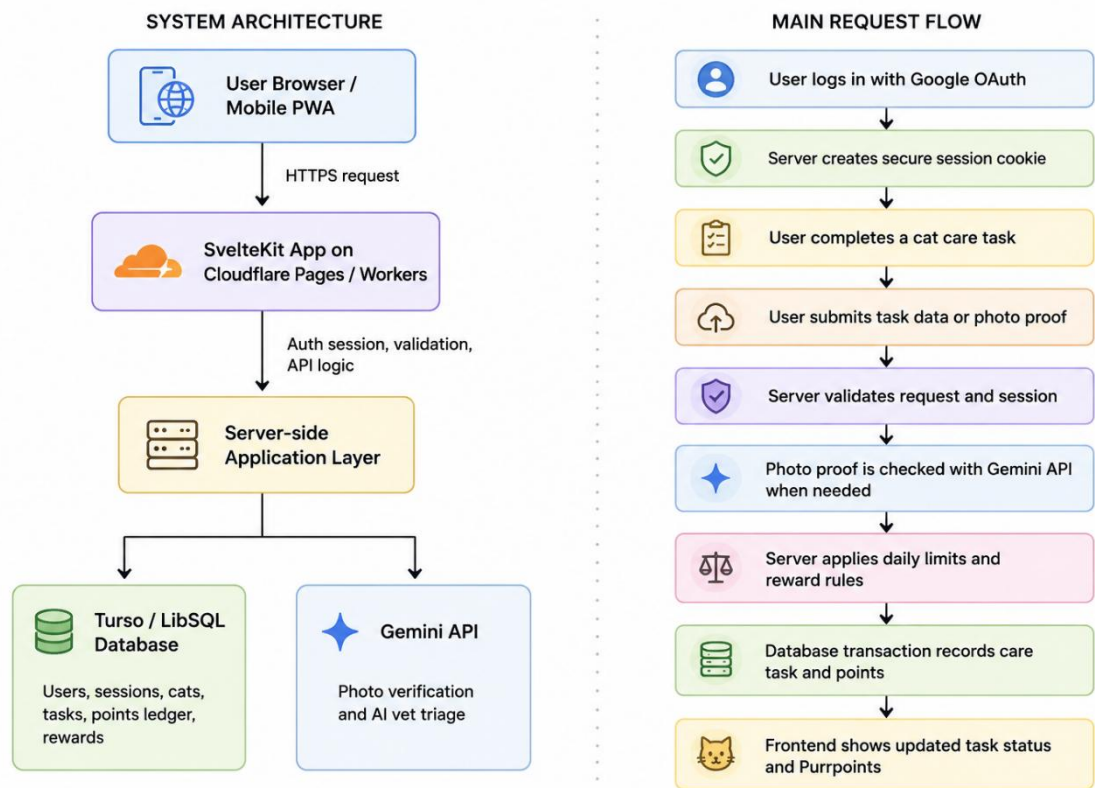
Layer	Technology
Runtime & Development	• Runtime / Package Manager: Bun • Bundler / Dev Server: Vite • Hosting Platform: Cloudflare Pages
Testing & Code Quality	• E2E Testing: Playwright • Unit Testing: Vitest • Linting & Formatting: ESLint, Prettier, Svelte Check (svelte-check)

5. Technical Architecture

Describe how the pieces of your system fit together. Include a diagram if possible (even a simple hand-drawn one), and explain the key components and how data/requests flow between them.

Purrward uses a SvelteKit-based full-stack architecture deployed on Cloudflare Pages and Workers. The frontend handles the mobile-first user interface, onboarding, task tracking, cat profile screens, rewards pages, and AI vet chat interface. The backend handles authentication, request validation, photo verification, point calculation, database writes, and protected API routes.

The system is designed so the client never controls sensitive values such as user identity, point amount, task limits, or verification status. Those decisions are handled by the server through authenticated API requests.



Key components about architecture:

1. **Frontend Client**
The frontend is built with SvelteKit 5, TypeScript, TailwindCSS, and supporting UI libraries. It provides the user-facing app experience, including onboarding, care tracking, rewards, cat profiles, and vet help.
2. **Authentication Layer**
Google OAuth 2.0 is used for login. The server manages the session using secure cookies, so the client does not need to store sensitive authentication tokens.
3. **Backend/API Layer**
SvelteKit server routes on Cloudflare Workers handle protected requests, validation, task completion, AI verification, reward calculation, and database access.
4. **Database Layer**
Turso with Drizzle ORM stores user accounts, sessions, cat information, task history, rewards, and points data. Database writes are handled server-side to keep the points system trustworthy.
5. **AI Verification and Vet Triage**
Gemini API is used for photo-based care verification and basic AI vet triage. This helps reduce fake task submissions and gives users a first layer of cat health guidance.
6. **Security Layer**
The system uses server-side validation, secure sessions, daily limits, upload validation, and parameterized database queries. This protects the app from fake point claims, unsafe uploads, and unauthorized access.

6. Testing Matrix

Optional section, but this will improve your marks on technical execution. List the manual test cases you ran to verify your project works as intended.

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Google OAuth Login	Click Sign in with Google and complete authentication.	User is authenticated and redirected to the home page with an active session.	Login completed successfully and user session was created.	Pass
Complete Care Task	Select a care task (e.g. Feeding) and mark it as completed.	The task is recorded and the daily progress is updated.	Task was saved and displayed as completed.	Pass
Photo Verification	Upload a photo as proof of completing a care task.	The server validates the image and Gemini verifies the submitted proof.	Photo was processed successfully and verification result was returned.	Pass
Purrpoints Reward	Complete a verified care task.	User receives Purrpoints according to the reward rules.	Purrpoints were added and displayed in the user account.	Pass
Daily Reward Limit	Repeat the same reward-earning task multiple times in one day.	Points stop increasing after the daily limit is reached.	Daily reward limit was enforced correctly.	Pass
AI Vet Triage	Submit a cat health question through the AI Vet feature.	AI returns a relevant cat care or triage response.	AI generated an appropriate	Pass

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
			response for the submitted question.	
Mobile Responsive UI	Open the application on a mobile device or responsive browser.	Pages remain readable and navigation works correctly on smaller screens.	UI adapted correctly to mobile screen sizes.	Pass
Invalid Input Handling	Submit invalid or incomplete form data.	Server rejects the request and displays an appropriate error message.	Invalid input was rejected without affecting application stability.	Pass

8. Future Improvements

If you had another week, what would you build or improve next? Show the judges you thought beyond the deadline.

☒ Build a more complete coupon and partner reward system

The current reward system can be expanded into real redeemable coupons, partner offers, or local pet shop rewards. This would make Purrrpoints more useful beyond the demo.

☒ Improve cat profile and multi-cat support

The app can support deeper cat profiles, including age, health notes, feeding preferences, medicine schedules, and different routines for each cat. This would help users who care for more than one pet or community cat.

☒ Add stronger community cat features

Future versions can include shared care logs, colony tracking, location-based feeding records, and collaboration tools for shelters or community caregivers.

☒ Improve AI verification accuracy

The photo verification flow can be improved with clearer task-specific prompts, better edge case handling, and safer fallback review when the AI result is uncertain.

☒ Add notification reminders

Push notifications or email reminders can help users remember feeding, water, litter, medicine, and other recurring care tasks.

☒ Add more detailed analytics

Users could see weekly care streaks, missed tasks, task history, and progress summaries to understand their cat care habits over time.

9. Tools You Used

List any tools that helped you build the project (e.g. AI coding assistants, design tools, recording software, etc.)

Development

1. Kiro: IDE and coding agent used for difficult implementation, debugging, and security-related work.
2. Codex: coding agent used to help implement, review, and improve the codebase.
3. Antigravity: coding assistant used for smaller fixes and backup development support.
4. GitHub: used for source code hosting and repository management.
5. Aikido: security test.

Infrastructure and Hosting

1. Cloudflare: serverless hosting and deployment platform.
2. Name.com: used for the project domain.

Art and Design

1. Ibis Paint: used by the artist to draw original cat assets.
2. Affinity by Canva: used for logo design and editing.
3. Canva: used for asset cropping and minor visual edits.
4. Favicon Generator: used to generate the project favicon.
5. ffmpeg: used to convert artist PNG assets into optimized WebP files.

Video and Media

1. CapCut Desktop: used for demo video editing, subtitles, and video text.
2. Xiaomi Redmi 14C built-in screen recorder: used to record mobile demo footage.
3. Youtube: used for demo video link.

Planning and Collaboration

1. ChatGPT: used for idea refinement, project planning, report check, and UI/UX feedback.
2. Google Drive: used to store and share project assets from the artist.
3. WhatsApp: used for team communication and asset coordination.
4. Microsoft Word: Report writing.

11. Learnings & Takeaways

What did the team learn, technically or otherwise, from building this project?

#hackthekitty was our second hackathon together, and it taught us a lot about turning an idea into a working product under time pressure. As a two-person team, we gained a better understanding of software architecture, user experience, and how to turn an initial concept into a complete product within a limited timeframe.

As the team leader and developer, I learned a great deal about SvelteKit, Turso, Drizzle ORM, authentication, server-side logic, and AI integration. This project also showed me that AI can be used for more than chatbots. We explored photo verification and AI vet triage while learning that sensitive logic such as authentication, points, and reward calculations must always remain on the server.

My teammate was responsible for the artwork and also acted as the primary design and idea reviewer throughout development. One of our biggest challenges was integrating the artwork into the application. While creating the illustrations was straightforward, implementing dynamic avatar assets introduced more complexity than we

expected. After several iterations, we decided to use pre-built assets for the MVP so we could deliver a stable and complete product within the hackathon deadline.

Overall, the project showed us that building a polished product is about more than writing code. Good planning, clear communication, continuous design reviews, and making practical trade-offs were just as important as the technical implementation.

12. Acknowledgments

Optional: credit any mentors, open-source projects, APIs, or datasets that helped along the way.

The team would like to thank the #hackthekitty 2026 organizers for hosting a hackathon that encouraged creative solutions for improving cat welfare through technology.

We also acknowledge the open-source projects and platforms that made this project possible, including SvelteKit, Tailwind CSS, Drizzle ORM, Turso, Cloudflare, Playwright, and Vitest.

Our sincere thanks go to the hackathon sponsors and supporters: Aikido for providing security testing resources, Kiro for the agentic development credits, and Temporal for supporting the event.

We also thank Google for the Gemini API, which powers the AI photo verification and AI vet triage features, as well as Google OAuth for secure user authentication.

Finally, special thanks to our teammate for creating the original cat artwork, reviewing ideas throughout development, shaping the UI direction, assisting with testing and project coordination, and for introducing her friend's cat, which became the star of our demo video. The demo also features the background music "Lofi Daydreams" by Rayky, used from the CapCut audio library.

Submission Checklist

- ☒ Video demo (HD or at least 720p)
<https://www.youtube.com/watch?v=G7h5uTeLtiY>
- ☒ README.md (prerequisites, run instructions, configuration)
- ☒ Project report (this document, in documentation/)
- ☒ Source code in src/
- ☒ No unrelated files, executables, auto-generated code, or package folders (e.g. node_modules/, build/, dist/, vendor/)